1501116 & 15011001 Computer Programming

Chapter 7 Iteration Statement: Loop Part 2

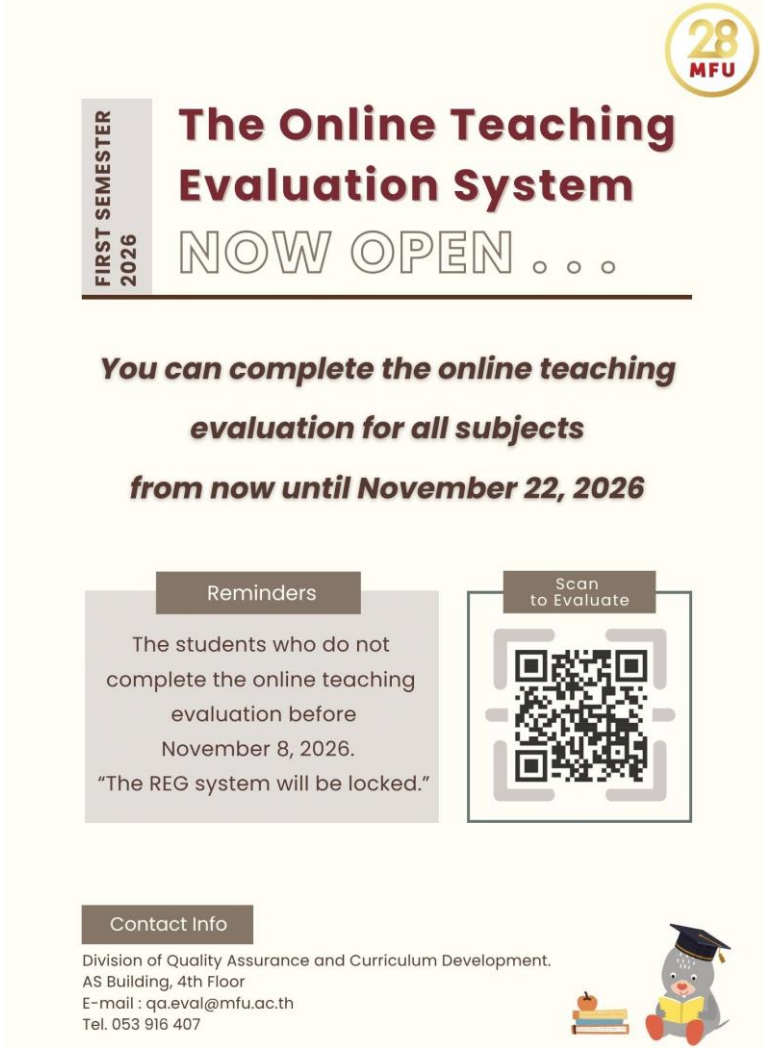
Academic Year 2026/01
School of Applied Digital Technology, Mae Fah Luang University

Revised version by: *Patcharaporn Panwong*
Paweena Suebsombut
Sujitra Arwatchananukul

Original version by: *Surapong Uttama*

Lesson Plan

Week	Weekly Summary
Week 10 (5-9 Oct 20206)	**Chapter 7:** Iteration Statement: Loop Part 2
Week 11 (12-16 Oct 2026)	**Chapter 8:** Methods
Week 12 (19-23 Oct 2026)	**Chapter 9:** Array
Week 13 (26-30 Oct 2026)	**Chapter 10:** Class and Object
Week 14 (2-6 Nov 2026)	Group Project Implementation
Week 15 (9-13 Nov 2026)	**Group Project Presentation** ★★★
Week 16 (16-20 Nov 2026)	**Group Project Presentation** ★★★
Final Exam (Thursday 26 November 2026, 09:00 - 12:00)	



Link:
https://evaloutcome.mfu.ac.th/login

Outline

- Review
- Posttest Loop Statement
  - do ... while
- Infinite Loops
- Jump statements
  - break
  - continue
- Nested Loops

Review

3 Categories of Control Statements (C#)

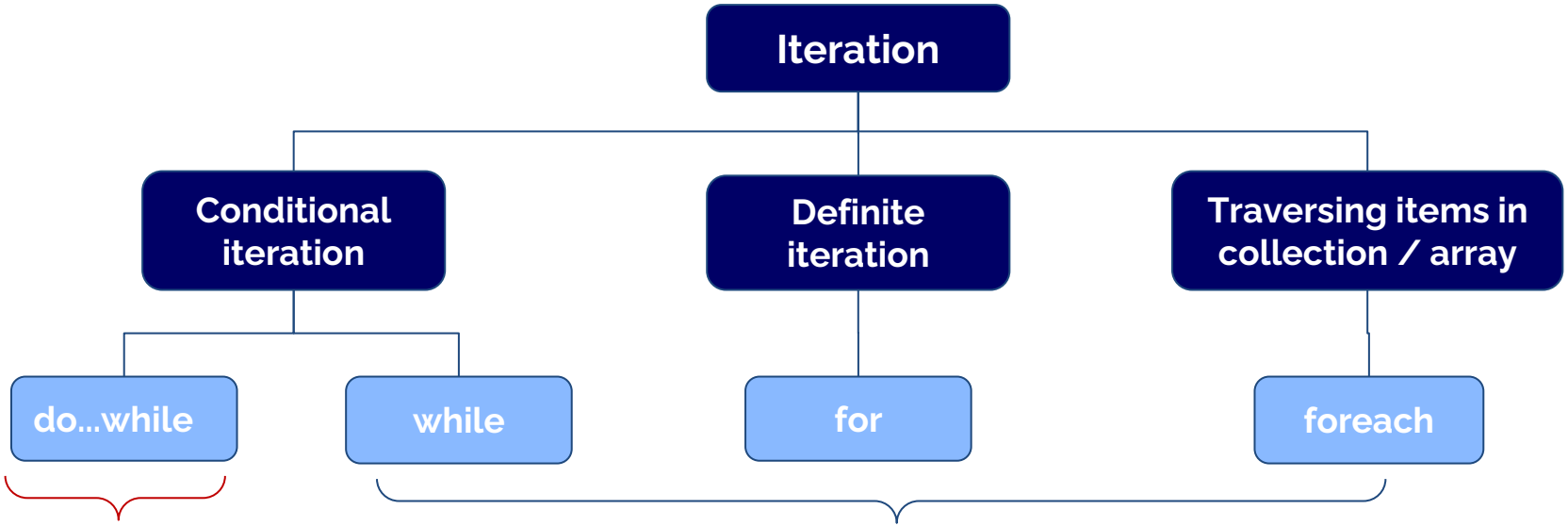
- Conditional/Selection statements
  This consists of **if**, **else**, **switch**, and **case** branching.

- Iteration/Loop statements
  This consists of **do**, **for**, **foreach**, and **while** looping.

- Jump statements
  This consists of **break**, **continue**, **return**, and **goto** statements.

More details about Iteration + Jump: Microsoft
Content Reference: Microsoft, freeasphosting

Kinds of Iteration Statements



Posttest Loop

```
do
{

} while ( condition )
```

```
while ( condition )
{

}
```

Pretest Loop

```
for ( initialization; condition; updation)
{

}
```

```
foreach ( Data_Type  variable_name  in Collection/array )
{

}
```

Self study: collection in C#

Kinds of Iteration Statements

1. **do…while**
   - Check a condition *after starting each loop*; loops if it's true

```
do
{

} while ( condition )
```

2. **while**
   - The most basic loop
   - Check a condition *before starting each loop*; loops if it's true

```
while ( condition )
{

}
```

3. **for**
   - Most common loop structure
   - A loop structure that contains *three separate statements*
   - Check a condition *before starting each loop*; loops if it's true

```
for ( initialization; condition; updation)
{

}
```

4. **foreach**
   - Automatically loop for a collection or an array of items.

```
foreach ( Data_Type  variable_name  in
Collection/array )
{

}
```

Components of Loop

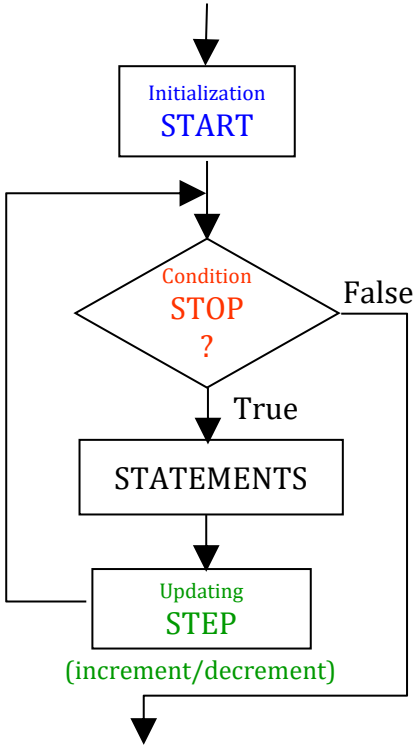
Loop's components

When repeating anything, a loop needs:

1. A **starting** point: "**Initialization step**" => **START**
2. A **decision** for continuing: "**Condition**" => **STOP**
3. A **runner** to control the loop: "**Counter step**" => **STEP**

(An "update" step for the next iteration)

Pretest Loops



- Check condition to stop before taking action
- If it is true, the loop action(s) is executed;
- if it is false, the loop is terminated.

```
while ( condition )
{

}
```

```
for ( initialization; condition; updation)
{

}
```

```
foreach ( Data_Type variable_name in Collection/array )
{

}
```

while ←→ for

Result / Output

```
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
```

```
int i = 1;
while(i<=10)
{
    Console.WriteLine("Hi");
    i++;
}
```

```
for( int i=1;i<=10;i++)
{
    Console.WriteLine("Hi");
}
```

for ←→ while

Result / Output

```
10
9
8
7
6
5
4
3
2
1
```

```
for(int i=10; i>=1; i-- )
{
    Console.WriteLine(i);
}
```

```
int i = 10;
while(i>=1)
{
    Console.WriteLine(i);
    i--;
}
```

Sample: Tracing a Loop

Ex1.

```
int i = 1;
while(i<=3)
{
    Console.WriteLine("Hi");
    i++;
}
```

Round	i	Result
1	1	Hi
2	2	Hi
3	3	Hi
4	4	

If the result of checking condition is false, the loop is terminated.

* animated series

Sample: Tracing a Loop

Ex2.

```
int i = 1, sum=0;
while(i<=3)
{
    sum+=i;
    i++;
}
```

Round	i	sum
1	1	1
2	_____	_____
3	3	_____
4	_____	

Sample: Tracing a Loop

Ex3.

```
int k;

for(k=1 ; k<=3 ; k++)
{
      if(k%2==1)
      {
         Console.WriteLine(k);
      }

}
```

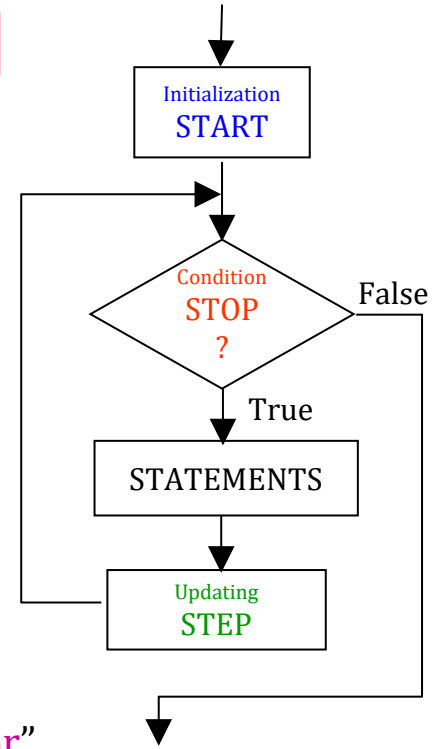
Round	k	Result
1	1	1
2	_____	_____
3	3	_____
4	4	

Posttest Loop Statements

- do...while

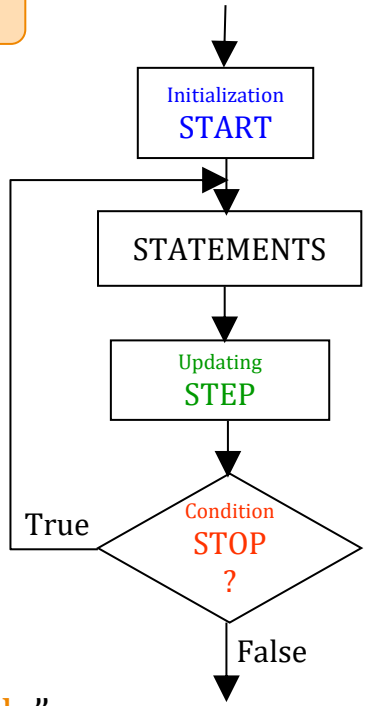
Components of Loops

Pretest



- Loop "while , for"
- Check BEFORE repeat

Posttest

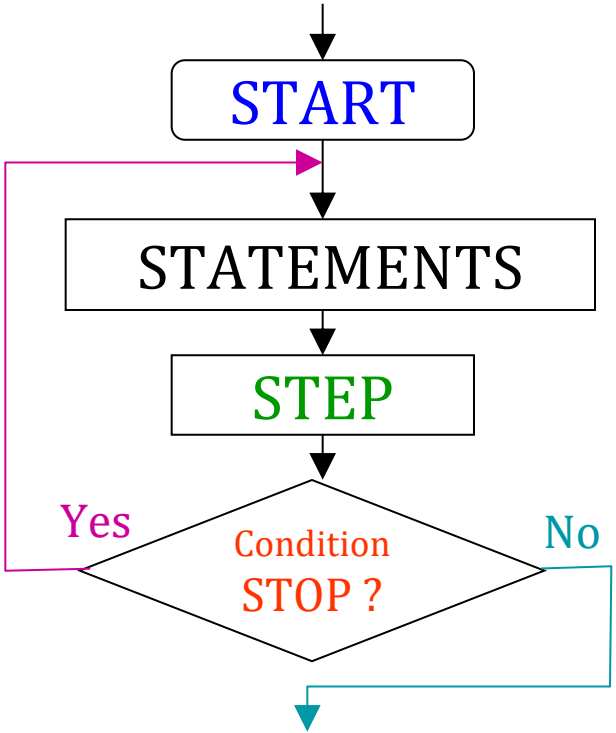


- Loop "do…while"
- Check AFTER repeat

do...while

The Posttest Loop

do...while



<code syntax>

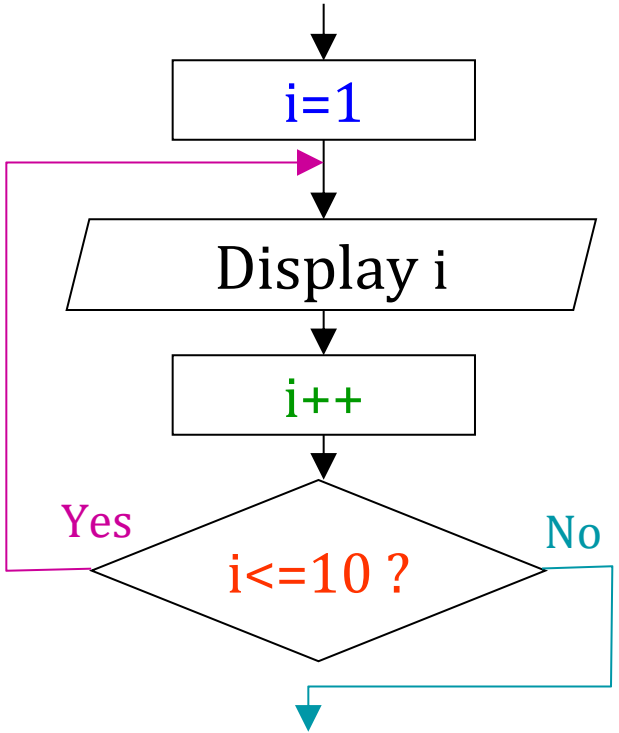
```
START;

do
{
    STATEMENTS;

    STEP; (increment/decrement)

} while(CONDITION) ;
```

do...while -- example 1



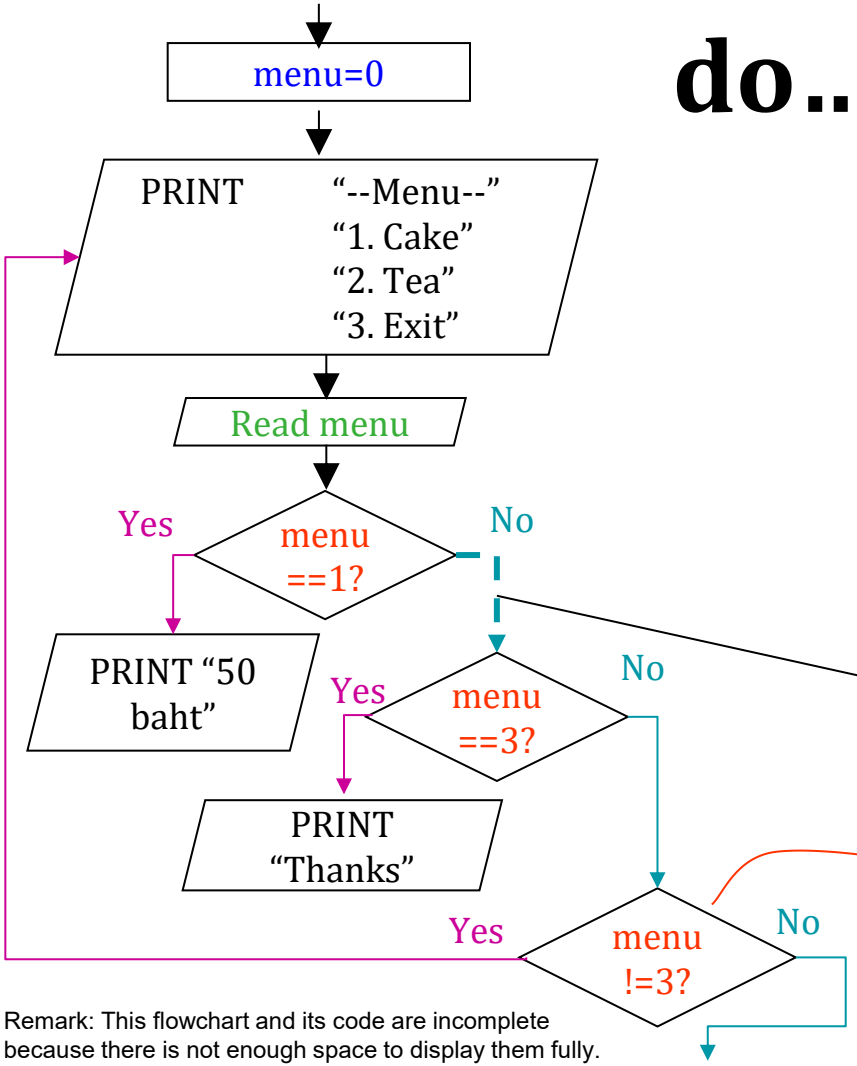

```
int i = 1;

do
{
    Console.WriteLine(i);

    i++;

} while(i<=10) ;
```

do...while -- example 2




```
int menu=0;
do
{
    Console.WriteLine("--Menu-- ……");
    menu=int.Parse(Console.ReadLine());
    if(menu==1){
        /* ask for payment */
        Console.WriteLine("50 baht");
    }else if (……){
        ……
    }else{
        Console.WriteLine("Thanks");
    }
} while(menu!=3) ;
```

Remark: This flowchart and its code are incomplete because there is not enough space to display them fully.

Pretest and Posttest Loops

Pretest

```
int i = 1;
while(i<=10)
{
   Console.WriteLine("Hi ");
   i++;
}
```

Posttest

```
int i = 1;
do
{
    Console.WriteLine("Hi ");
    i++;
} while(i<=10);
```

Common Loops' Errors

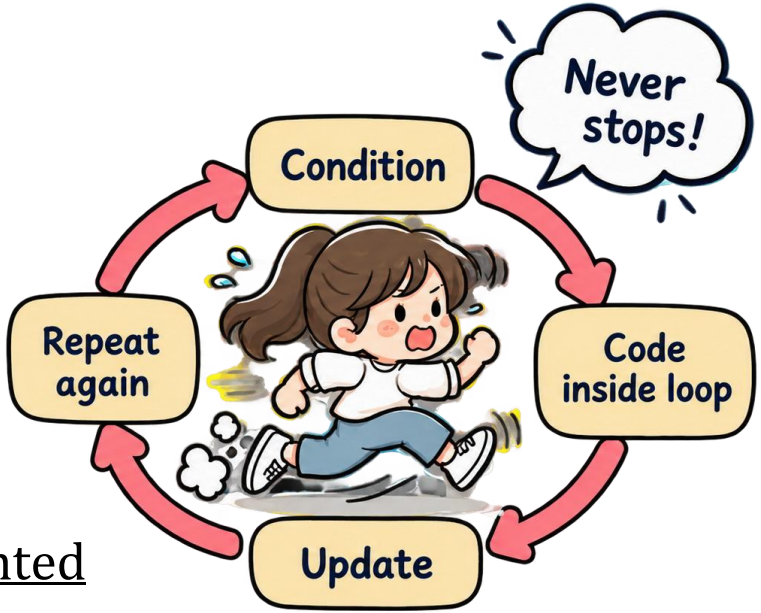
- Forget START or CONTINUE or STEP
- Extra ; after *while* and *for*
- Forget ; after *do…while*
- Use , instead of ; in *for*
- Forget { }

Infinite Loop



Infinite Loop

- Infinite / forever / perpetual loops
- Loop that runs without stopping
- Causes
  - Missing of START, STOP or STEP
  - Unordered START, STOP and STEP
  - *Intention*
  - etc.
- Generally, infinite loops should be <u>prevented</u>



Infinite Loops: *Missing*

```
for(int i=1; i<=10 ;  )
{
    Console.WriteLine("Hi ");
}
```



Multiple Choice

Infinite Loops: *Missing*

```
int i = 1;

while(i<=10)

{

    Console.WriteLine("Hi ");

    STEP;

}
```



Infinite Loops: *Unordered*

```
int i = 1;
while (i <= 10)
{
    Console.WriteLine("Hi ");
    i--;
}
```




```
int i = 1;
while (i > 0)
{
    Console.WriteLine("Hi ");
    i++;
}
```



Infinite Loops: *Intention*

```
for (; ; )
{
    Console.WriteLine("Hi ");
}
```



```
while (true)
{
    Console.WriteLine("Hi ");
}
```



```
do
{
    Console.WriteLine("Hi ");
} while (true);
```





Jump Statements

- break
- continue

Jump Statements

Jump statements change the execution of a loop

1. break

- Break out of the loop entirely
- Some loops need a break before their STOP
- Infinite loops need a break

2. continue

- Break out of this iteration of the loop and move on to the next

“break” statement

- It breaks the current loop.

```
int i = -5;
while( i<=5 )
{
    Console.WriteLine(10.0/i);
    i++;
}
```

Result / Output

```
-2
-2.5
-3.33333333333333
-5
-10
∞
10
5
3.33333333333333
2.5
2
```



*** break => if i = 0**

```
int i = -5;
while( i<=5 )
{
    if( i==0 ){
        break;
    }
    Console.WriteLine(10.0/i);
    i++;
}
```

Result / Output

```
-2
-2.5
-3.33333333333333
-5
-10
```

“break” statement

```
int i = -5;
while( i<=5 )
{
        if( i==0 ){
            break;
        }
        Console.WriteLine(10.0/i);
        i++;
}
```

???

Will it be the same?

```
int i = -5;
while( i<=5 )
{
        if( i==0 ){
            i=5;
        } else{
            Console.WriteLine(10.0/i);
        }
        i++;
}
```

Result / Output

```
-2
-2.5
-3.33333333333333
-5
-10
```

“continue” statement

- Start the next round of the current loop immediately
- Skip the next statements in the loop

```
for(int i=1; i<=10; i++)
{
    Console.Write(i + " ");
}
```

Result / Output

```
1 2 3 4 5 6 7 8 9 10
```

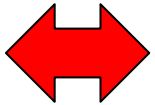
```
for(int i=1; i<=10; i++)
{
        if(i==5){
            continue;
        }
        Console.Write(i + " ");
}
```

Result???

Result / Output

“continue” statement

```
for(int i=1; i<=10; i++)
{
        if(i==5){
            continue;
        }
        Console.Write(i + " ");
}
```



```
for(int i=1; i<=10; i++)
{
        if(    ){
            Console.Write(i + " ");
        }
}
```

“continue” is not frequently found.
It can be replaced by another simple code.

break vs continue

```
int i;
for(i=-5; i<=5; i++)
{
        if( i==0 ){
            break;
        }
        Console.Write(i + " ");
}
```

stop when i = 0

Result:

```
int i;
for(i=-5; i<=5; i++)
{
        if( i==0 ){
            continue;
        }
        Console.Write(i + " ");
}
```

skip when i = 0

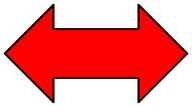
Result:

continue + “while” or “do...while”

```
for(int i=1; i<=10; i++)
{
        if( i==5 ){
            continue;
        }
        Console.Write(i + " ");
}
```

Result / Output

```
1 2 3 4 6 7 8 9 10
```



```
int i=1;
while( i<=10 )
{
        if( i==5 )
        {
            i++;
            continue;
        }
        Console.Write(i + " ");
        i++;
}
```

Be careful that the “continue” in *“while”* or *“do...while”* loop must be after the STEP.
Otherwise, the loop will be infinite.

"continue" makes the program jump to the CONTINUE condition immediately and skip all statements after it.

```
for(int i=1; i<=10 ; i++)
{
        statement1;
        continue;
        statement2;
}
```

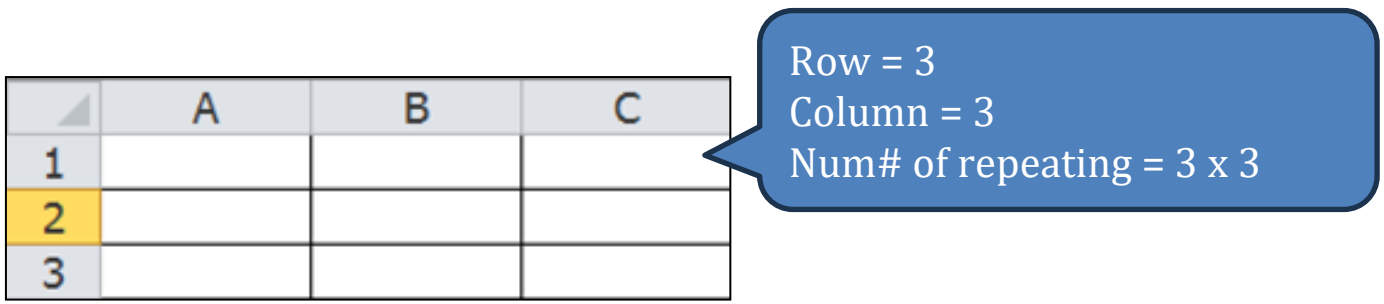
```
int i=1;
do
{
        statement1;
        i++;
        continue;
        statement2;
} while( i<=10 );
```

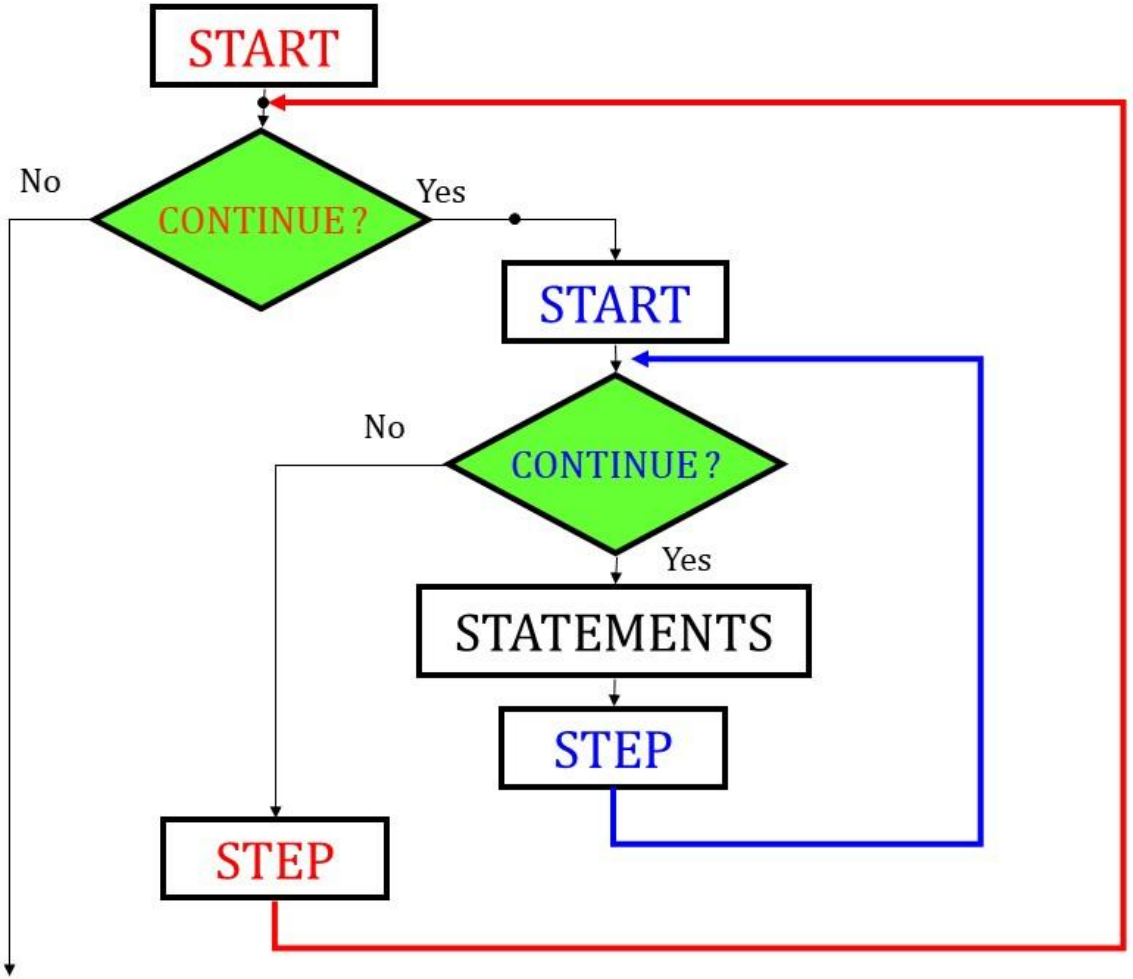
```
int i=1;
while( i<=10 )
{
        statement1;
        i++;
        continue;
        statement2;
}
```

Nested Loop

Nested Loop

- Loops within loops
- Loop is processed from inside to outside
- The total number of loops **=** the number of loops multiplied
- **If** there are **two** nested loops, think as **row** and **column**




START
No
CONTINUE ?
Yes
START
No
CONTINUE ?
Yes
STATEMENTS
STEP
STEP

Nested Loop

Sample #1

- Write C# program to display the same result as expected.

Expected result

```
xxxxx
xxxxx
xxxxx
```

```
int row, col;
for(row=1 ; row<=3 ; row++)
{
        for(col=1; col<=5; col++)
        {
            Console.Write("x");
        }
        Console.WriteLine();
}
```

Sample #2

- Write C# program to display the same result as expected.

Expected result

```
xxxxx
xxxxx
xxxxx
xxxxx
xxxxx
```

```
int r, c;
for(r=1 ; r<=5 ; r++)
{
        for(c=1; c<=5; c++)
        {
                Console.Write("x");
        }
        Console.WriteLine();
}
```

Remark:
- r stands for row
- c stands for column

Sample #3

- Write C# program to display the same result as expected.

Expected result

```
x
xx
xxx
xxxx
xxxxx
```

```
int r, c;
for(r=1 ; r<=5 ; r++)
{
        for(c=1; c<=r; c++)
        {
                Console.Write("x");
        }
        Console.WriteLine();
}
```

Sample #4

- Write C# program to display the same result as expected.

Expected result

```
xxxxx
xxxx
xxx
xx
x
```

```
int r, c;
for(r=5 ; r>=1 ; r--)
{
        for(c=1; c<=r; c++)
        {
                Console.Write("x");
        }
        Console.WriteLine();
}
```

Exercise #1

- Write C# program to display the same result as expected.

1

Expected result

```
11111
1111
111
11
1
```

2

Expected result

```
1
12
123
1234
12345
```

Exercise #1.1

1

Expected result

```
11111
1111
111
11
1
```

```
int r, c;
for(r=5; r>=1; r--)
{
    for(                    )
    {
        Console.Write("1");
    }
    Console.WriteLine();
}
```

Fill in the Blanks

Exercise #1.2

2

Expected result

```
1
12
123
1234
12345
```

<<Outer loop>> r	<<Inner loop>> c
1 2 … 5	1 to 1 1 to 2 … 1 to 5

```
int r, c;
for(r=1; r<=5; r++)
{
    for(c=1; [      ]; c++)
    {
        Console.Write ([  ]) ;
    }
    Console.WriteLine();
}
```

Summary

- How many loop statements are there?
- What are differences between Pretest and Posttest loops?
- How do we break a loop?
- Can we put a loop inside another loop?


Q&A

Appendix

Nested Loop

- If you understand how the basic nested loop works, please look at the next three slides.

Nested Loop

- We can put a loop inside another loop.

```
int i= 1, j = 5;
while(i<=3)
{
        while(j>=1)
        {
                Console.WriteLine(i + “    ” + j);
                j--;
        }
        i++;
        j=5;
}
```

Result

```
1   5
1   4
1   3
1   2
1   1
2   5
2   4
2   3
2   2
2   1
3   5
3   4
3   3
3   2
3   1
```

Nested Loop

- We can put a loop inside another loop.

```
int i= 1, j = 5;
while(i<=3)
{
        while(j>=1)
        {
                Console.WriteLine(i + “    ” + j);
                j--;
        }
        i++;
        j=5;
}
```

Same example with the previous slide, but you **remove this command**. So, the **result change**.

Result

```
1   5
1   4
1   3
1   2
1   1
```

Nested Loop

Assign value, j=5, one time (outside while loop)

```
int i= 1, j = 5;
while(i<=3)
{
    while(j>=1)
    {
        Console.WriteLine(i + “    ” + j);
        j--;
    }
    i++;
}
```

Result

```
1   5
1   4
1   3
1   2
1   1
```

≠

```
int i= 1, j = 5;
for(i=1; i<=3; i++)
 {
        for(j=5; j>=1; j--)
        {
                Console.WriteLine(i + “    ” + j);
        }
}
```

Assign j=5 every time for new iteration of main loop

Result

```
i   j
1   5
1   4
1   3
1   2
1   1
2   5
2   4
2   3
2   2
2   1
3   5
3   4
3   3
3   2
3   1
```